leetcode 4

921使括号串有效的最小添加

C++

```cpp
class Solution {
public:
    int minAddToMakeValid(string s) {
        stack<char> stk;
        for (char c:s) {
            if (!stk.empty() && c==')' && stk.top()=='(') {
                stk.pop();
                continue;
            }
            stk.push(c);
        }
        return stk.size();
    }
};
```

1021仅删除外层括号

C++

```cpp
class Solution {
public:
    string removeOuterParentheses(string s) {
        string ans;
        stack<char> stk;
        int left=0,right;
        for (int i=0;i<s.size();i++) {
            if (!stk.empty() && s[i]==')' && stk.top()=='(') stk.pop();
            else stk.push(s[i]);
            if (stk.empty()) {
                right=i;
                ans+=s.substr(left+1,right-left-1);
                left=right+1;
            }
        }
        return ans;
    }
};
```

1190反转括号内的字符串

C++

```cpp
//给出一个字符串 s（仅含有小写英文字母和括号）。请你按照从括号内到外的顺序，逐层反转每对匹配括号中的字符串，并返回最终的结果。
//注意，您的结果中 不应 包含任何括号。
class Solution {
public:
    string reverseParentheses(string s) {
        stack<int> stk;  // 存储左括号索引
        for (int i = 0; i < s.size(); i++) {
            if (s[i] == '(') {
                stk.push(i);
            } else if (s[i] == ')') {
                int left = stk.top(); stk.pop();   // 匹配的左括号索引
                // 提取括号内的子串 [left+1, i-1]
                string inner = s.substr(left + 1, i - left - 1);
                reverse(inner.begin(), inner.end()); // 反转内部
                // 将整个括号段（包括括号）替换为反转后的内部字符串
                s.replace(left, i - left + 1, inner);
                // 调整 i 的位置：替换后字符串长度变化，i 指向替换后的最后一个字符
                i = left + inner.size() - 1;
            }
        }
        return s;  // 此时已经没有括号了
    }
};
```

- stack<int> stk;
  - stack 是一个容器适配器，提供后进先出（LIFO）的数据结构。
  - int 表示栈中存放的元素类型是 int，这里存的是字符的下标（索引）。
  - 常用成员函数：
    - push(x)：将元素 x 压入栈顶。
    - pop()：移除栈顶元素（无返回值）。
    - top()：返回栈顶元素的引用（不删除）。
    - empty()：判断栈是否为空。
  - 作用：每当遇到左括号 (，就把它的位置（下标）存进栈。遇到右括号) 时，栈顶就是匹配的左括号位置。这样就能知道要反转的区间。

- for (int i = 0; i < s.size(); i++)
  - 遍历字符串 s 的每个字符。
  - s.size() 返回字符串长度（size_t 类型，通常与 int 比较没问题，但初学者可以理解为整数）。
- int left = stk.top(); stk.pop();
  - stk.top() 返回栈顶元素（左括号下标），赋值给 left。
  - stk.pop() 弹出栈顶，表示这一对括号已经处理过了。
- string inner = s.substr(left + 1, i - left - 1);
  - substr 是 string 的成员函数，用于提取子串。
  - 第一个参数：起始位置（下标）。
  - 第二个参数：要提取的长度。
  - 这里 left+1 是左括号右边第一个字符的位置，i-1 是右括号左边最后一个字符的位置。长度 = (i-1) - (left+1) + 1 = i - left - 1。
- reverse(inner.begin(), inner.end());
  - reverse 是 <algorithm> 头文件中的函数，用于反转一个序列。
  - 参数是两个迭代器：起始迭代器和结束迭代器（指向最后一个元素之后）。
  - inner.begin() 返回指向 inner 第一个字符的迭代器。
  - inner.end() 返回指向末尾后一个位置的迭代器。
- s.replace(left, i - left + 1, inner);
  - replace 是 string 的成员函数，用于替换子串。
  - 第一个参数：起始位置。
  - 第二个参数：要替换的长度。
  - 第三个参数：用来替换的新字符串。
  - 这里起始位置是 left，长度是 i - left + 1，即从 left 到 i 总共多少个字符（包括 (和)）。我们用反转后的 inner 替换整个 (...) 段。

3120统计同时有其大小写形式的字母数量

C++

```cpp
class Solution {
public:
    int numberOfSpecialChars(string word) {
        int mask[2]{};
        for (char c:word) mask[c>>5&1]|=1<<(c&31);
        return popcount(1u*(mask[0]&mask[1]));
    }
};
```

这段代码巧妙地运用了位图和ASCII 编码规律，用 O(n) 时间、O(1) 空间就完成了统计。我们逐步拆解它背后的原理。

1. 整体思路：用两个整数当“字母签到表”

程序维护了一个数组 int mask[2]，两个 int 通常至少 32 位，足以装下 26 个字母：

- mask[0] — 用来记录大写字母是否出现过（哪个字母出现，就把对应位置成 1）
- mask[1] — 用来记录小写字母是否出现过

最后，把两个整数的位做按位与（&）：如果同一个字母的大小写都出现过，对应位就会是 1。统计结果中有多少个 1，就是答案。

---

2. 为什么 c>>5 & 1 能区分大小写?

来看几个字母的 ASCII 码二进制（只展示低 7 位）：

字符 ASCII 二进制 (7位) 高 2 位 (bit6,5) c>>5 & 1

'A' 65 100 0001 10 (2) 0

'Z' 90 101 1010 10 (2) 0

'a' 97 110 0001 11 (3) 1

'z' 122 111 1010 11 (3) 1

- c >> 5：将字符右移 5 位，相当于只保留最高 2 位（除以 32 取整）。
- 大写字母 65~90 除以 32 等于 2（二进制 10）
- 小写字母 97~122 除以 32 等于 3（二进制 11）
- & 1：再取最低位，大写得到 0，小写得到 1。

因此 c>>5 & 1 就是大小写的天然索引：0 代表大写，1 代表小写。 这就解释了为什么要用 mask[2]。

---

3. 为什么 c & 31 能得到字母序号?

31 的二进制是 0001 1111（5 个 1）。任何数和 31 按位与，相当于只保留低 5 位，也就是该数除以 32 的余数。

观察字母的低 5 位：

字符 ASCII 低5位 c & 31 字母序号

'A' 65 = 64 + 1 1 1

'B' 66 = 64 + 2 2 2

… … … …

'Z' 90 = 64 + 26 26 26

'a' 97 = 96 + 1 1 1

'z' 122= 96 + 26 26 26

大小写字母的低 5 位完全相同，而且正好是 1~26。

于是 c & 31 就是字母的序号（A/a=1，B/b=2，…，Z/z=26）。

---

4. 如何“签到”字母？1 << (c & 31)

1 << n 就是把 1 左移 n 位，得到一个只有第 n 位（从 0 开始）是 1、其余位是 0 的数。

例如 'A' 的 c & 31 是 1，1 << 1 得到二进制 ...0000 0010（第 1 位为 1）。

然后 mask[idx] |= 1 << n 将这个位“置 1”，标记该字母出现过。

整个过程：

C++

```
for (char c : word)
    mask[c >> 5 & 1] |= 1 << (c & 31);
```

- 遇到大写字母，更新 mask[0] 的对应位
- 遇到小写字母，更新 mask[1] 的对应位

5. 找到同时存在的大小写字母

```C++
mask[0] & mask[1]
```

- 按位与：只有当 mask[0] 和 mask[1] 的同一位都是 1 时，结果的该位才为 1。
- 这正好表示：某字母的大小写都出现过。

比如字母 F (序号 6)：如果 F 和 f 都在单词里，那么 mask[0] 和 mask[1] 的第 6 位都是 1，与结果的第 6 位就是 1。

---

6. 数出有多少个这样的字母：popcount

popcount（population count）函数统计一个整数二进制表示中 1 的个数。

- 在 C++20 中是 std::popcount()
- GCC/Clang 中常用 __builtin_popcount()
- 前面乘以 1u 只是为了确保传给 popcount 的是无符号整数（类型规范）

最终 popcount 返回的值，就是同时拥有大小写形式的字母个数。

总结：为什么这段代码又短又快?

1. 位图 将 26 个字母存在两个整数里，操作极快。
2. 利用 ASCII 编码规律，用简单的 >>5 和 &31 代替 if / isupper / islower 等分支或查表，无跳转、无额外内存。
3. 最后用一个 按位与 + popcount，一次得到交集和计数。
4. 整体 O(n) 时间，O(1) 额外空间，只有一次遍历和少量位运算，CPU 执行效率极高。

希望这个解释能帮你完全看清这段代码的每一个细节。如果还想了解 popcount 硬件指令层面的原理，也可以继续问我。

1717消除ab/ba后的最大得分

```C++
//给你一个字符串 s 和两个整数 x 和 y 。你可以执行下面两种操作任意次。
```

```cpp
//删除子字符串 "ab" 并得到 x 分。比方说，从 "cabxbae" 删除 ab ，得到 "cxbae" 。
//删除子字符串"ba" 并得到 y 分。比方说，从 "cabxbae" 删除 ba ，得到 "cabxe" 。
//请返回对 s 字符串执行上面操作若干次能得到的最大得分。
class Solution {
public:
    int maximumGain(string s, int x, int y) {
        string stk;   // 用 string 模拟栈
        string temp;
        int ans = 0;
        if (x >= y) {
            for (char c : s) {
                if (!stk.empty() && c == 'b' && stk.back() == 'a') {
                    stk.pop_back();
                    ans += x;
                } else {
                    stk.push_back(c);
                }
            }
            for (char c : stk) {   // 剩余字符顺序不变，可正常遍历
                if (!temp.empty() && c == 'a' && temp.back() == 'b') {
                    temp.pop_back();
                    ans += y;
                } else {
                    temp.push_back(c);
                }
            }
        } else {
            for (char c : s) {
                if (!stk.empty() && c == 'a' && stk.back() == 'b') {
                    stk.pop_back();
                    ans += y;
                } else {
                    stk.push_back(c);
                }
            }
            for (char c : stk) {
                if (!temp.empty() && c == 'b' && temp.back() == 'a') {
                    temp.pop_back();
                    ans += x;
                } else {
                    temp.push_back(c);
                }
            }
        }
        return ans;
    }
};
class Solution {
```

```cpp
public:
    int maximumGain(string s, int x, int y) {
        char a = 'a', b = 'b';
        if (x < y) {
            swap(x, y);
            swap(a, b);
        }

        int ans = 0, cnt1 = 0, cnt2 = 0;
        for (char c : s) {
            if (c == a) {
                cnt1++;
            } else if (c == b) {
                if (cnt1) {
                    ans += x;
                    cnt1--;
                } else {
                    cnt2++;
                }
            } else {
                ans += min(cnt1, cnt2) * y;
                cnt1 = 0;
                cnt2 = 0;
            }
        }
        ans += min(cnt1, cnt2) * y;
        return ans;
    }
};
```

20有效括号

C++

```cpp
class Solution {
public:
    bool isValid(string s) {
        int n = s.size();
        if (n % 2 == 1) {
            return false;
        }
        unordered_map<char, char> pairs = {
            {')', '('},
            {']', '['},
            {'}', '{'}
        };
```

```
        //这是一个哈希表（unordered_map），将右括号映射到对应的左括号。写成 {')', '('} 表示：键是 )，值是 (。这样当我们遇到右括号时，可以立刻查到它应该匹配哪个左括号。
        stack<char> stk;
        for (char ch: s) {
            if (pairs.count(ch)) {
            //pairs.count(ch) 检查 ch 是否在哈希表中。如果 ch 是右括号（)、]、}），count 返回 1，进入这个分支；否则是左括号，进入 else。
                if (stk.empty() || stk.top() != pairs[ch]) {
                //如果栈顶的左括号 stk.top() 不等于当前右括号期望的匹配值 pairs[ch] → 非法（类型不匹配）。
                    return false;
                }
                stk.pop();
            }
            else {
                stk.push(ch);
            }
        }
        return stk.empty();
    }
};
```

1249移除无效的括号

C++

```
class Solution {
public:
    string minRemoveToMakeValid(string s) {
        stack<int> stk;
        for (int i=0;i<s.size();i++) {
            if (islower(s[i])) continue;
            if (!stk.empty() && s[i]==')' && s[stk.top()]=='(') stk.pop();
            else stk.push(i);//注意s[i]为字母的情况
        }
        while (!stk.empty()) {
            s.erase(stk.top(),1);
            stk.pop();
        }
        return s;
    }
};
```

150逆波兰表达式求值（重点在vector<string>上）

C++

```cpp
class Solution {
public:
    int evalRPN(vector<string>& tokens) {
        stack<int> stk;
        for (string c:tokens) {
            if (c=="+" || c=="-" || c=="*" || c=="/") {
                //tokens中的每一项其实是一个string，所以要用双引号和stoi函数
                int op2=stk.top();
                stk.pop();
                int op1=stk.top();
                stk.pop();
                char op=c[0];
                switch (op) {
                    case '+': stk.push(op1+op2); break;
                    case '-': stk.push(op1-op2); break;
                    case '*': stk.push(op1*op2); break;
                    case '/': stk.push(op1/op2); break;
                }
            }
            else stk.push(stoi(c));
        }
        return stk.top();
    }
};
```

394按数字数量赋值括号内字符串后的结果

C++

```cpp
class Solution {
public:
    string decodeString(string s) {
        string ans;
        stack<int> stk;//用来记录即将被解码的字符串的重复次数
        stack<int> par;//用来记录与目前右括号配对的左括号的位置
        int len=s.size();
        for (int i=0;i<len;i++) {
            ans.push_back(s[i]);
            if (isdigit(s[i])) {
                int t=s[i]-'0';
                while (i>0 && isdigit(s[i-1])) {
                    t+=stk.top()*10;
                    stk.pop();
```

```
                }//注意数字不一定是个位数，因此要算上全部数字
                stk.push(t);
            }
            else if (s[i]=='[') par.push(ans.size());
            else if (s[i]==']') {
                string temp=ans.substr(par.top(),ans.size()-par.top()-1);//记录所需要重复的部分
                string paste;
                for (int j=0;j<stk.top();j++) paste+=temp;//进行若干次重复
                ans.replace(par.top()-2,ans.size()-par.top()+2,paste);//而后进行替换
                //一定要注意起止位置！！！
                par.pop();
                stk.pop();//将使用过的数字和左括号替换掉
            }
        }
        return ans;
    }
};
```

1106解析布尔表达式

C++

```
class Solution {
public:
    bool parseBoolExpr(string expression) {
        vector<char> stk;//用于记录除逗号外的字符
        vector<char> op;//用于记录操作符
        int len=expression.size();
        for (int i=0;i<=len;i++) {
            if (i<len && expression[i]==',') continue;//跳过逗号
            else if (i<len && (expression[i]=='!' || expression[i]=='|' || expression[i]=='&')) op.push_back(expression[i]);//记录操作符，注意操作符不压入stk
            else if (i==len && !op.empty() || expression[i]==')') {
                char oprt=op.back();
                op.pop_back();
                switch (oprt) {
                    case '!': {
                        char back=stk.back();
                        while (!stk.empty() && stk.back()!='(') stk.pop_back();
                        stk.pop_back();
                        stk.push_back(back=='t'?'f':'t');
                    } break;
```

```
                    case '|': {
                        int flag=0;
                        while (!stk.empty() && stk.back()!='(') {
                            char back=stk.back();
                            stk.pop_back();
                            if (back=='t') flag=1;
                        }
                        stk.pop_back();
                        stk.push_back(flag==0?'f':'t');//用flag确定新压入的字符
                    } break;
                    case '&': {
                        int flag=1;
                        while (!stk.empty() && stk.back()!='(') {
                            char back=stk.back();
                            stk.pop_back();//依旧要全部弹出
                            if (back=='f') flag=0;
                        }
                        stk.pop_back();//注意把最后一个左括号弹出
                        stk.push_back(flag==1?'t':'f');
                    } break;
                }
            }
            else if (i<len) stk.push_back(expression[i]);//注意对长度len进行保护!
        }
        return stk.back()=='f'?false:true;
    }
};
//对表达式从左向右进行阅读
//对除了逗号以外的操作符进行压栈
//如果遇到右括号就反向计数，将所有元素录入vector并读取操作
//分操作类型得到结果而后压栈，注意需要将原本的所有内容出栈，同时避免穿透括号的需要
```

726根据化学式记录各个原子的数量并按字典序输出

C++

```
class Solution {
public:
    string countOfAtoms(string formula) {
        // 用 vector<pair<string,int>> 模拟栈
        // string 存原子名（左括号哨兵也存在这里），int 存数量
        vector<pair<string, int>> stk;//通过vector pair建立元素与数量的关系
        int i = 0;
        int n = formula.size();
```

```
        while (i < n) {
            // ——————————————————————————————————
            // 情况①：大写字母 → 解析原子名 + 后面的数字
            // ——————————————————————————————————
            if (isupper(formula[i])) {
                string name = "";
                // 先读首字母（一定是大写）
                name += formula[i++];
                // 再读后续小写字母（原子名可能多个字母，如 "Mg"）
                while (i < n && islower(formula[i])) name += formula[i++];
                // 读后面的数字（注意：可能是多位数，如 "Fe12"）
                //  【易错】只读一位会导致 "H12" 解析成 1 而不是 12
                int num = 0;
                while (i < n && isdigit(formula[i]))num = num * 10 +
(formula[i++] - '0');
                // 没有数字时默认为 1
                if (num == 0) num = 1;
                stk.push_back({name, num});
            // ——————————————————————————————————
            // 情况②：左括号 → 压入哨兵，数量填 0 占位
            // ——————————————————————————————————
            }
            else if (formula[i] == '(') {
                stk.push_back({"(", 0});
                i++;
            }
            // ——————————————————————————————————
            // 情况③：右括号 → 读倍数，弹栈到哨兵，全部乘倍数后压回
            // ——————————————————————————————————
            else {  // formula[i] == ')'
                i++;  // 跳过 ')'
                // 读括号后面的倍数（同样注意多位数）
                int num = 0;
                while (i < n && isdigit(formula[i]))num = num * 10 +
(formula[i++] - '0');
                //  【易错】括号后没有数字时默认倍数为 1，不能留 0
                if (num == 0) num = 1;
                // 把括号内的元素全部弹出，暂存到 temp
                //  【易错】弹栈顺序是反的，但因为后面只做乘法，顺序不影响结果
                vector<pair<string, int>> temp;
                while (stk.back().first != "(") {
                    temp.push_back(stk.back());
                    stk.pop_back();
                }
                // 弹出哨兵 "("
                stk.pop_back();
                // 乘以倍数后重新压回主栈
```

```
                for (auto& p : temp)stk.push_back({p.first, p.second * num});
            }
        }
        // ——————————————————————————————————————————
        // 后处理：用 map 合并同名原子并自动完成字典序排序
        // 【注意】map<string,int> 的 key 是按字典序自动排列的
        //         省去了手写 qsort / std::sort 的麻烦
        // ——————————————————————————————————————————
        map<string, int> cnt;
        for (auto& p : stk)cnt[p.first] += p.second;
        // 拼接最终答案
        string ans = "";
        for (auto& p : cnt) {
            ans += p.first;
            // 数量为 1 时不输出数字
            if (p.second > 1)ans += to_string(p.second);
        }
        return ans;
    }
};
```

队列！！

649Dota2参议院

C++

```
//Dota2 的世界里有两个阵营：Radiant（天辉）和 Dire（夜魇）
//Dota2 参议院由来自两派的参议员组成。现在参议院希望对一个 Dota2 游戏里的改变作出决定。他们以一个基于轮为过程的投票进行。在每一轮中，每一位参议员都可以行使两项权利中的 一项：
//剥夺一名参议员的权利：一名参议员可以使另一名参议员在本轮及所有后续轮次中失去所有权利。
//宣布胜利：如果参议员发现有权利投票的参议员都是 同一个阵营的 ，他可以宣布胜利并决定在游戏中的有关变化。
//给你一个字符串 senate 代表每个参议员的阵营。字母 'R' 和 'D'分别代表了 Radiant（天辉）和 Dire（夜魇）。然后，如果有 n 个参议员，给定字符串的大小将是 n。以轮为基础的过程从给定顺序的第一个参议员开始到最后一个参议员结束。这一过程将持续到投票结束。所有失去权利的参议员将在过程中被跳过。
//假设每一位参议员都足够聪明，会为自己的政党做出最好的策略，你需要预测哪一方最终会宣布胜利并在 Dota2 游戏中决定改变。输出应该是 "Radiant" 或 "Dire" 。
class Solution {
public:
    string predictPartyVictory(string senate) {
        int len=senate.size();
        int r_exe=0,d_exe=0,r_num=0,d_num=0,r_enum=0,d_enum=0;
```

```
        //exe表示被操作的位置，num表示总数，enum表示被清除的数量
        for (int i=0;i<len;i++) {
            if (senate[i]=='R') r_num++;
            else d_num++;
        }
        int i=0;
        while (r_enum<r_num && d_enum<d_num) {//确保删除数小于议员数
            if (senate[i]=='R') {
                d_enum++;
                d_exe=i;//从当前开始向后寻找，可以避免一些麻烦
                while (senate[d_exe]!='D') d_exe=
(d_exe+1)%senate.size();//确保永远在范围内
                senate.erase(d_exe,1);
                i++;
                if (d_exe<i) i--;//如果操作位置在前面，要记得将i--
            }
            else {
                r_enum++;
                r_exe=i;
                while (senate[r_exe]!='R') r_exe=(r_exe+1)%senate.size();
                senate.erase(r_exe,1);
                i++;
                if (r_exe<i) i--;
            }
            if (i>=senate.size()) i=0;//可能需要不止一轮操作
        }
        return senate.back()=='R'?"Radiant":"Dire";
    }
};
```

950显示一张牌后将第一张放回末尾的方式下能够正序显示牌面的序列

C++

```
class Solution {
public:
    vector<int> deckRevealedIncreasing(vector<int>& deck) {
        deque<int> dq;
        sort(deck.begin(),deck.end(),greater());
        int len=deck.size();
        for (int i=0;i<len;i++) {
            if (i==0) dq.push_back(deck[i]);
            else {
                int temp=dq.back();
                dq.pop_back();
                dq.push_front(temp);
```

```
                dq.push_front(deck[i]);
            }
        }
        vector<int> ans;
        for (int i:dq) ans.push_back(i);
        return ans;
    }
};
```

739返回往后更高温度的距离天数的数组

C++

```
class Solution {
public:
    vector<int> dailyTemperatures(vector<int>& temperatures) {
        stack<int> stk;
        int len=temperatures.size();
        for (int i=0;i<len;i++) {
            if (i==0 || temperatures[i]<temperatures[stk.top()])
stk.push(i);
            else {
                while (!stk.empty() &&
temperatures[i]>temperatures[stk.top()]) {
                    temperatures[stk.top()]=stk.top()-i;
                    stk.pop();
                }
                stk.push(i);
            }
        }
        for (int i=0;i<len;i++) {
            if (temperatures[i]<0) temperatures[i]*=(-1);
            else temperatures[i]=0;
        }
        return temperatures;
    }
};
```

768能完成排序的最多块数组

C++

```
//给你一个整数数组 arr 。将 arr 分割成若干 块 ，并将这些块分别进行排序。之后再连接起
来，使得连接的结果和按升序排序后的原数组相同。
```

```cpp
//返回能将数组分成的最多块数?
class Solution {
public:
    int maxChunksToSorted(vector<int>& arr) {
        stack<int> stk;
        for (int i:arr) {
            if (stk.empty() || i>stk.top()) stk.push(i);
            else {
                int max=stk.top();
                stk.top();
                while (!stk.empty() && stk.top()>i) stk.pop();
                stk.push(max);
            }
        }
        return stk.size();
    }
};
```

核心教训:

与其追踪"何时需要合并"（事件驱动，容易漏），不如维护一个始终满足不变量的数据结构（状态驱动，每步都对）。

第二个程序之所以简洁正确，是因为它找到了问题的不变量：排好序后，每个块的最大值必须构成递增序列。单调栈天然维护这个不变量，无需分类讨论各种峰谷情形。

选题时的一般原则： 当你发现自己在写"发现 X 情况就特殊处理"的分支时，往往意味着还没找到正确的抽象。正确的抽象通常让每步操作都是同质的，不需要大量条件分支。

2054寻找两个最好的不重叠活动

C++

```cpp
//给你一个下标从 0 开始的二维整数数组 events ，其中 events[i] = [startTimei, endTimei, valuei] 。第 i 个活动开始于 startTimei ，结束于 endTimei ，如果你参加这个活动，那么你可以得到价值 valuei 。你 最多 可以参加 两个时间不重叠 活动，使得它们的价值之和 最大 。
//请你返回价值之和的 最大值 。
//注意，活动的开始时间和结束时间是 包括 在活动时间内的，也就是说，你不能参加两个活动且它们之一的开始时间等于另一个活动的结束时间。更具体的，如果你参加一个活动，且结束时间为 t ，那么下一个活动必须在 t + 1 或之后的时间开始。
class Solution {
public:
    int maxTwoEvents(vector<vector<int>>& events) {

        // ranges::sort 是 C++20 引入的排序算法，支持"投影"参数
```

```cpp
        // 第一个参数：待排序容器；第二个参数：比较器，{} 表示使用默认的 < 运算符；第三个参数：投影函数，提取用于比较的"键"，这里取每个活动的 e[1]（结束时间）

        ranges::sort(events, {}, [](auto& e) { return e[1]; });

        // pair<int,int> 是标准库提供的"捆绑两个值"的结构体；访问方式：p.first 取第一个值，p.second 取第二个值
        // 若需捆绑三个及以上的值，使用 tuple<int,int,int>，用 get<0>(t) 等访问
        // 这里每个元素存 (结束时间, 截至该时间的历史最大价值)
        //
        // 该栈维护两个单调性（从栈底到栈顶）：
        //   - 结束时间严格递增（支持二分查找）
        //   - 价值严格递增（保证查到即最优，无需再扫描）
        // 哨兵 {0,0} 含义：结束时间为 0，价值为 0，代表"不选第一个活动"的基准情况，保证了后续 -1 操作不会越界，也覆盖了"只选一个活动"的情况

        vector<pair<int,int>> stk = {{0, 0}};
        int ans = 0;

        // auto& 中的 & 表示引用，直接操作原数据而非拷贝
        // 不加 & 写 auto cur 则每次循环都拷贝一份 vector<int>，有额外开销
        // 只读不写时更严格的写法是 const auto&

        for (auto& cur : events) {

            // ranges::lower_bound 在有序序列中找第一个 >= 目标值的位置
            // 返回值是迭代器，行为类似指针，指向找到的元素
            //
            // 四个参数含义：
            //   stk           : 在此容器中查找
            //   cur[0]        : 目标值，即当前活动的开始时间
            //   {}            : 比较器，使用默认 <
            //   &pair<int,int>::first : 投影（成员指针）
            //
            // &pair<int,int>::first 是"成员指针"语法
            // 作用：告诉算法比较时只取每个 pair 的 .first（结束时间）
            // 这里的 & 不是取地址，而是取"成员指针"，把"访问某成员"这个动作作为参数
            //
            // lower_bound 返回第一个 >= cur[0] 的位置
            // -1 退一步，得到最后一个结束时间 < cur[0] 的元素
            // 由于栈内价值单调递增，该元素的 .second 就是最优的第一个活动价值

            auto one = ranges::lower_bound(stk, cur[0], {}, &pair<int,int>::first) - 1;

            // one 是迭代器（类似指针），访问其指向的对象成员必须用 ->
            // 不能用 . ，因为 . 只用于直接访问对象本身的成员
            // one->second 等价于 (*one).second（先解引用，再访问成员）
```

```
            // 当 one 指向哨兵 {0,0} 时，one->second = 0。即"不选第一个活动"，此时只统计当前一个活动的价值

            ans = max(ans, one->second + cur[2]);

            // 只有当前活动价值严格大于栈顶价值时才入栈
            // 否则它不可能成为最优的"第一个活动"，直接丢弃
            // 这一步维护了栈内价值的单调递增性
            //
            // emplace_back 是原地构造：直接将参数转发到容器内部构造对象
            // push_back 是先构造临时对象再拷贝/移动进去，多一步开销
            // 写法对比：
            //   stk.emplace_back(cur[1], cur[2]);    // 原地构造，推荐
            //   stk.push_back({cur[1], cur[2]});     // 构造临时 pair 再移入

            if (cur[2] > stk.back().second) stk.emplace_back(cur[1], cur[2]);
        }
        return ans;
    }
};
```

3113边界元素是最大值的子数组数

C++

```
//给你一个 正 整数数组 nums 。
//请你求出 nums 中有多少个子数组，满足子数组中 第一个 和 最后一个 元素都是这个子数组中的 最大 值。
class Solution {
public:
    long long numberOfSubarrays(vector<int>& nums) {
        long long ans=nums.size();
        stack<pair<int,int>> stk;
        stk.emplace(INT_MAX,0);
        for (int i:nums) {
            while (i>stk.top().first) stk.pop();
            if (i<stk.top().first) stk.emplace(i,1);
            else ans+=stk.top().second++;
        }
        return ans;
    }
};
//通过维护一个底大顶小的栈，我们可以确保是最大值，而且通过维护其中的出现次数正确添加数量
```